DESARROLLO DE APLICACIONES II

Respuesta a M6 Ambiente — publicación por AMQP

Módulo 9 · Core

20 de octubre de 2026

Contenido

1 · ¿Dónde se valida `sourceModule` contra el token, publicando por AMQP? 3
2 · Usuario AMQP por módulo, permisos, vhost, amqps 3
3 · `maxAttempts` y la cadena 15 s → 1 m → 5 m → 15 m 4
4 · ¿El sobre llega tal cual, o le agregan campos? 4
5 · ¿`muni.inbox` siempre tiene al menos una cola bindeada? 5
6 · `q.ambiente` : ¿clásica o quorum? ¿Con qué argumentos? 5
7 · `eventProcessingFailed` y `eventRejected` 5
8 · UUID y `causationId` 6
9 · Lo pendiente desde agosto y septiembre 6
Ambientes y credenciales 7
Resumen de lo que les cambia el código 7

> **Lo primero, porque les cambia el código:** hoy, publicando por AMQP, **no validamos que `sourceModule` coincida con nadie**. El 403 que describimos es del endpoint HTTP y sólo de ahí. Por la cola, cualquiera puede declarar cualquier `sourceModule`. Está explicado en el punto 1, es un agujero nuestro y lo vamos a cerrar.
>
> Y lo segundo: **las URLs del 13/10 no existen.** Seguimos sin deploy. Los puntos 2 y el de credenciales siguen sin respuesta real, y no les queremos dar una fecha que ya incumplimos una vez.

Antes que nada: gracias por asumir lo de `occurredAt` y `sourceModule: "ambiente"`. Las dos están bien. El `occuredAt` de la tabla era un typo nuestro.

---

1 · ¿Dónde se valida `sourceModule` contra el token, publicando por AMQP?

Hoy no se valida en ningún lado.

Hay dos caminos de ingesta y no están parejos:

Camino	Autenticación	¿Verifica `sourceModule`?
`POST /api/v1/events`	Bearer JWT	**Sí.** Si el sobre declara otro módulo → 403
Publicar en `muni.inbox`	ninguna	**No.** Se procesa tal cual viene

El worker que consume `core.inbox` valida la **forma** del sobre y nada más. No recibe ningún token y no compara el `sourceModule` con nada.

Así que, respondiendo lo concreto que preguntan: **no va token en un header, no hay validación por usuario AMQP, y sí existe el endpoint HTTP** (`POST /api/v1/events`, con `Authorization: Bearer <token de módulo>`), que es el único camino donde hoy la identidad se verifica de verdad.

Qué hacemos con esto. Es un hallazgo nuestro, no un detalle de configuración. Lo cerramos con usuarios AMQP por módulo y verificando el `user-id` del mensaje contra el `sourceModule` del sobre — que es el mecanismo que RabbitMQ ya trae para esto. Pero necesita el broker desplegado, así que queda atado al punto 2.

Qué les conviene hacer mientras tanto. Sigan con AMQP, que es el camino recomendado y no les va a cambiar: cuando se cierre, lo que cambia es que su usuario AMQP tendrá que ser el suyo, no `guest`. El sobre no cambia.

---

2 · Usuario AMQP por módulo, permisos, vhost, amqps

No existen todavía. No hay un RabbitMQ compartido desplegado.

Hoy cada equipo corre el suyo local, con `guest/guest` en el vhost `/`, sin TLS.

Cuando exista, esto es lo que proponemos y no debería sorprenderlos:

	Valor previsto
Usuario	uno por módulo (`ambiente`), nunca `guest`
`configure`	vacío — las colas y los bindings los declara el Core
`write`	sólo `muni.inbox`
`read`	sólo `q.ambiente`
Vhost	uno por ambiente (`/test` , `/prod`), no prefijos de cola
TLS	`amqps://` en los dos ambientes

Lo de `configure` vacío es a propósito y se relaciona con el punto 6: ustedes declaran en modo pasivo, que es exactamente lo correcto.

---

3 · `maxAttempts` y la cadena 15 s → 1 m → 5 m → 15 m

No son lo mismo, y se pueden desalinear. Son dos cosas separadas:

- **La cadena de esperas** es global del Core: `RETRY_DELAYS=15,60,300,900` .
- **`maxAttempts`** es por suscripción. El de ustedes está en **4**.

La espera del intento *n* se toma de la posición *n* de la cadena, y **si se pasa, se queda en la última**:

```python
def _delay_for(attempt: int) -> int:
    delays = settings.retry_delays or [5]
    return delays[min(max(attempt, 1), len(delays)) - 1]
```

Entonces, respondiendo lo que preguntan:

- `maxAttempts = 4` → 15 s, 1 m, 5 m, 15 m. Coincide con la cadena. Es su caso.
- `maxAttempts = 6` → 15 s, 1 m, 5 m, 15 m, **15 m, 15 m**. No falla, repite el último escalón.
- `maxAttempts = 2` → 15 s, 1 m, y a la DLQ. Se usan los dos primeros nomás.

Agotados los intentos, la entrega queda `DEAD` y aparece en `GET /api/v1/dlq` .

---

4 · ¿El sobre llega tal cual, o le agregan campos?

El cuerpo es exactamente el sobre. No agregamos `receivedAt` ni nada. Lo nuestro viaja en **headers AMQP**, no en el body:

Header	Qué trae
`x-core-event-id`	el `eventId` , también como `message_id`
`x-core-target-module`	a quién va dirigida esa copia
`x-core-attempt`	número de intento, arranca en 1
`x-core-trace-id`	para correlacionar con nuestra auditoría
`x-core-last-error`	sólo en reintentos: el error anterior

Pero hay dos detalles que sí les pueden cambiar el código, porque el cuerpo se re-serializa, no se reenvía byte a byte:

Los `null` desaparecen. Serializamos con `exclude_none` . Si mandan `"causationId": null` , el consumidor recibe el campo **ausente**. Mandarlo nulo y omitirlo son indistinguibles del otro lado — ver punto 8.

`occurredAt` **se normaliza, conservando el offset.** Lo que entra como `2026-10-20T09:15:30.5-03:00` sale como `2026-10-20T09:15:30.500000-03:00`. El instante y la zona son los mismos; la representación no es idéntica. Si comparan strings de fecha, eso les va a fallar.

Un ejemplo real de lo que sale por la cola:

```
{
  "eventId": "3f6c1b7e-9d24-4a1f-9f2a-2b0f0c7d5e11",
  "eventType": "containerDamaged",
  "eventVersion": "1.0",
  "occurredAt": "2026-10-20T09:15:30.500000-03:00",
  "sourceModule": "ambiente",
  "correlationId": "8a1f0c22-5d3e-4b77-9c10-6e2b4a90f3d5",
  "data": { "containerId": "C-18" }
}
```

---

5 · ¿`muni.inbox` siempre tiene al menos una cola bindeada?

Sí, y su estrategia de `mandatory: true` es correcta. Pero con una condición que conviene que sepan.

`muni.inbox` es **fanout** y tiene siempre `core.inbox` bindeada. Las dos se declaran juntas, como infraestructura base, en el arranque del Core y antes de cualquier suscripción. Nunca hay un estado donde el exchange exista sin su cola.

La condición: eso lo declara el Core al arrancar. Si publican contra un broker donde el Core **nunca corrió**, el exchange directamente no existe y no van a recibir un `basic.return` sino un error de canal (404 NOT_FOUND).

Conviene que distingan los dos casos en el manejo de errores: `basic.return` = no hay a quién entregar (no debería pasar nunca); 404 de canal = el Core todavía no declaró la topología en ese broker.

Y `core.inbox` es **durable**: si el Core está caído, ustedes siguen publicando sin error y los mensajes esperan ahí. Nada se pierde.

---

6 · `q.ambiente`: ¿clásica o quorum? ¿Con qué argumentos?

Clásica y durable. No usamos quorum. Un solo argumento:

```
durable: true
x-dead-letter-exchange: muni.dlx
```

Sin `x-queue-type`, sin TTL, sin `x-max-length`.

Declarar en pasivo es lo correcto y es lo que queremos: quien define la cola es el Core, cuando se registra la suscripción. Si alguna vez un declare pasivo suyo falla con 404, es que la suscripción no está activa de nuestro lado, no que haya que crearla.

---

7 · `eventProcessingFailed` y `eventRejected`

Están de acuerdo: sáquenlos. De hecho son **nuestros**, no de ustedes.

```
eventProcessingFailed   owner=core   publica=[core]   escucha=[core]   recibidos=0
eventRejected           owner=core   publica=[core]   escucha=[]       recibidos=0
```

Nunca circuló ninguno de los dos. Son de una versión del diseño en la que el Core avisaba los fallos por evento, antes de que existieran la DLQ y la auditoría. Hoy eso ya está cubierto y mejor:

- `GET /api/v1/dlq` — lo que falló, con su motivo
- `GET /api/v1/dlq/{id}/audit` — quién reintentó qué y cuándo
- `GET /api/v1/events/{eventId}` — el estado de cada entrega, módulo por módulo

Los damos de baja del catálogo. Si los tenían declarados de su lado, sáquenlos.

---

8 · UUID y `causationId`

El validador NO es RFC estricto. Es `uuid.UUID()` de Python: no mira versión ni variante. Lo pasamos por casos concretos:

Entrada	Resultado
v4 canónico	acepta
v1, v7	acepta
nil (`00000000-...`)	acepta
variante inválida	**acepta**
sin guiones	acepta, y le pone los guiones
mayúsculas	acepta, normaliza a minúsculas
`{...}` con llaves	acepta, las saca
`urn:uuid:...`	acepta, saca el prefijo
`CORR-2026-00184`	**rechaza**

O sea: cualquier cosa que Python pueda interpretar como UUID entra, y **sale siempre normalizada** al 8-4-4-4-12 en minúsculas. Si generan v7 para que ordenen por tiempo, no hay problema.

Sobre `causationId` : omitirlo está perfecto, sigan así. Y no se compliquen, porque **da igual**: serializamos con `exclude_none` , así que si mandan `null` , el consumidor lo recibe ausente igual. Las dos formas son indistinguibles del otro lado. Omitirlo es lo que recomendamos, que es lo que ya hacen.

---

9 · Lo pendiente desde agosto y septiembre

Tres de las cuatro **no son nuestras**, y creemos que por eso quedaron sin respuesta tanto tiempo. Lo decimos para que no sigan esperándonos:

Catálogo de barrios con `neighborhoodId` estable. No es del Core. Nosotros no tenemos datos maestros: no sabemos de barrios, ciudadanos ni expedientes, sólo movemos sobres. Quien tenga el padrón territorial es el que tiene que publicarlo. Nuestra lectura es que le corresponde a Ciudadanos, pero es una suposición nuestra; conviene preguntarlo en el grupo general.

Si nuestra «zona» es su zona operativa. No tenemos ninguna «zona». El único tipo del catálogo con ese concepto es `zoneNotServiced` , **y es de ustedes** (owner `ambiente` , lo publican ustedes y no lo escucha nadie). Así que si hay dos definiciones de zona dando vueltas, ninguna es del Core.

Si alguien publica `notificationSent` . No. **No existe en el catálogo**: ningún módulo lo declara, ninguno lo escucha, nunca llegó uno. Si lo están esperando, hoy no va a venir de ningún lado.

La lista de eventos propios del Core. Esta sí es nuestra, y es corta: `eventProcessingFailed` y `eventRejected` , los dos del punto 7, los dos sin uso y que vamos a dar de baja. O sea: **después de eso, el Core no publica ningún evento de negocio.** Somos pasamanos, no productor.

Ambientes y credenciales

La fecha del 13/10 no se cumplió y no hay URLs. No hay deploy: ni del Core, ni de Postgres, ni de RabbitMQ. No está trabado en nosotros, es una decisión de equipo que no se tomó todavía, y ya hay tres módulos esperando lo mismo (ustedes, Obras y Tránsito).

No les damos otra fecha hasta que esté decidido dónde se despliega. Preferimos eso a incumplir una segunda.

Las credenciales, cuando existan, se entregan por privado y de a uno. Un secreto de máquina pegado en el chat del grupo queda en el historial de todos para siempre. Y aclaramos algo del punto 1: el secreto de módulo sirve para sacar el token de la API HTTP; el usuario de RabbitMQ será otra credencial distinta.

Mientras tanto pueden integrar local y no van a tirar nada a la basura. El Core levanta en dos comandos y el sobre, el exchange, la cola y los headers ya son los definitivos:

```
cd backend
.venv/bin/python -m app.seeds
.venv/bin/uvicorn app.main:app --reload
.venv/bin/python -m app.workers.inbox_worker   # el consumidor de core.inbox, va aparte
```

Swagger en `http://localhost:8000/docs`. Su usuario de dashboard es `ambiente@muni.uade.edu.ar` / `Cambiala123`.

Resumen de lo que les cambia el código

1. **Por AMQP no validamos identidad hoy.** Cuando lo cerremos, su usuario de RabbitMQ tendrá que ser el suyo. El sobre no cambia.
2. **Los `null` se caen del cuerpo** al re-serializar. Omitir es equivalente.
3. **`occurredAt` se normaliza**: no comparen strings de fecha.
4. **Distingan `basic.return` de 404 de canal**: el segundo es que el Core nunca declaró la topología en ese broker.
5. **`maxAttempts` y la cadena de esperas son independientes.** Con 4 coinciden.
6. **`eventProcessingFailed` y `eventRejected` se dan de baja.**
7. **`notificationSent` no existe**: si lo esperan, no va a llegar.